time series part 2

me

Table of contents

ENV SETUP IMPORTANT TO RUN ONCE **2**
MPS IMPORT . . . 3
REST OF DATASET IMPORTS AND READS . . . 5
DATA CLEANING . . . 6
PLOTS . . . 6

(ii) Answer **8**
EURO STOXX 50 . . . 8
VSTOXX . . . 8
ESI . . . 8
Harmonised Index of Consumer Prices . . . 8
EURO-AREA AAA Government Bond Rates . . . 8
Plots . . . 9

(iii) Construct z_{t-1}. **9**
3-Month Cutoff FUNCTION . . . 9
CUTOFF FUNCTION APPLY . . . 9
PCA ON GOVERNMENT BOND YIELD LEVELS . . . 12
WHY PC2? . . . 12
CALCULATE THE THREE MONTH PC2 CHANGE . . . 12
PREDICTOR VERIFICATION . . . 13

(iii) Answer **15**

(iv) Describe the Predictors. **15**
Summary Stats . . . 16

(iv) Answer **16**

(v) Estimate the Regression. **16**

(v) Answer **17**

(vi) Check the Residuals. **17**
REsid Check . . . 18

(vi) Residual & Serial Correlation Testing **19**

(vii) Inference with HAC Standard Errors. **19**

(vii) Answers **20**

(viii) Test individual and joint significance. **20**

(viii) Answer **21**

(ix) Robustness Check. **21**
(a) 21
(b) 23
(c) 24
(d) 24

(x) Interpret. **25**

ENV SETUP IMPORTANT TO RUN ONCE

needs v2tx.csv file in working directory to import properly

```
#install.packages(c("eurostat", "ecb", "quantmod", "sandwich", "lmtest", 'readr', 'readxl', 'patchwork'
#install.packages(
#  c("dplyr", "tidyr", "lubridate"),
#  repos = "https://cloud.r-project.org"
#)

library(readxl);library(car); library(lmtest); library(ggplot2); library(patchwork); library(eurostat);
```

```
Loading required package: carData
```

```
Loading required package: zoo
```

```

Attaching package: 'zoo'
```

```
The following objects are masked from 'package:base':

    as.Date, as.Date.numeric
```

```
Loading required package: xts
```

```
Loading required package: TTR
```

```
Registered S3 method overwritten by 'quantmod':
  method            from
  as.zoo.data.frame zoo
```

```

######################### Warning from 'xts' package ##########################
#                                                                             #
# The dplyr lag() function breaks how base R's lag() function is supposed to  #
# work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or       #
# source() into this session won't work correctly.                            #
#                                                                             #
# Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
# conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop           #
# dplyr from breaking base R's lag() function.                                #
#                                                                             #
# Code in packages is not affected. It's protected by R's namespace mechanism #
# Set `options(xts.warn_dplyr_breaks_lag = FALSE)` to suppress this warning.  #
#                                                                             #
###############################################################################
```

```
Attaching package: 'dplyr'
The following objects are masked from 'package:xts':

    first, last
The following object is masked from 'package:car':

    recode
The following objects are masked from 'package:stats':

    filter, lag
The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union

Attaching package: 'lubridate'
The following objects are masked from 'package:base':

    date, intersect, setdiff, union
Deleted .rds/.RData files and cache_list.json from /var/folders/y7/2x_6clks0ls11sgb_v6y6k2r0000gn/T//Rtm
```

MPS IMPORT

```
mps <- read.csv("../data/mps.csv")

monthly_mps <- mps %>%
  mutate(month = floor_date(as.Date(date), "month")) %>%
  group_by(month) %>%
  summarise(
    unstandardized = sum(as.numeric(unstandardized), na.rm = TRUE),
    .groups = "drop"
  ) %>%
  complete(
    month = seq.Date(min(month), max(month), by = "month"),
    fill = list(unstandardized = 0)
  ) %>%
  mutate(
    month_date = month,
    month = format(month, "%Y-%m")
  )

cat('Length of monthly_mps: \n')
```

```
Length of monthly_mps:
```

```
print(nrow(monthly_mps))
```

```
[1] 286
```

```
cat('sample period \n')
```

```
sample period
```

```
cat(strrep('*', 50), '\n')
```

```
**************************************************
```

```
cat('start period of sample: ', as.character(monthly_mps$month[1], '\n'))
```

```
start period of sample:  2002-01
```

```
cat('end period of sample: ', as.character(tail(monthly_mps$month,1), '\n'))
```

```
end period of sample:  2025-10
```

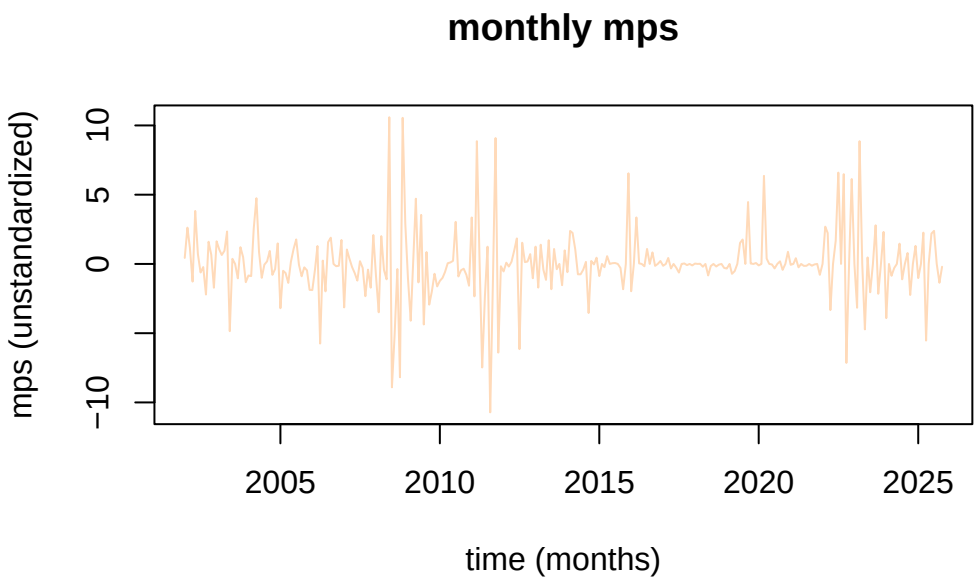
```
# append numeric cols to this list for simple plotting
numeric_cols <- monthly_mps[, sapply(monthly_mps, is.numeric), drop = FALSE]

# grabbed from last notebook
simple_summary <- sapply(numeric_cols, function(x) c(
  Mean = mean(x, na.rm = TRUE),
  Std_Dev = sd(x, na.rm = TRUE),
  Min = min(x, na.rm = TRUE),
  Max = max(x, na.rm = TRUE)
))

simple_summary
```

```
        unstandardized
Mean     -1.904071e-16
Std_Dev   2.495350e+00
Min      -1.071652e+01
Max       1.059272e+01
```

```
# plot
plot(
  x = monthly_mps$month_date,
  y = monthly_mps$unstandardized,
  type='l',
  main = 'monthly mps',
  xlab = 'time (months)',
  ylab = 'mps (unstandardized)',
  col = 'peachpuff',
  lwd=1
)
```



REST OF DATASET IMPORTS AND READS

STOXX from 2007 ->

```
# hicp from eurostat api
hicp <- get_eurostat("prc_hicp_manr", filters = list(geo = "EA20", coicop = "CP00"))
```

```
Table prc_hicp_manr cached at /var/folders/y7/2x_6clks0ls11sgb_v6y6k2r0000gn/T//RtmpY1W1qo/eurostat/be1
```

```
# esi using api func.
esi <- get_eurostat(id = "ei_bssi_m_r2", filters = list(indic = "BS-ESI-I",s_adj = "SA", geo="EA20"), t
```

```
Table ei_bssi_m_r2 cached at /var/folders/y7/2x_6clks0ls11sgb_v6y6k2r0000gn/T//RtmpY1W1qo/eurostat/d7fc
```

```
if ("time" %in% names(esi)) {
  esi <- esi %>% arrange(time)
} else if ("TIME_PERIOD" %in% names(esi)) {
  esi <- esi %>% arrange(TIME_PERIOD)
}

# yield curves for 6 different maturities
mats <- c("3M", "6M", "1Y", "2Y", "5Y", "10Y")
yc <- lapply(mats, function(m)
get_data(paste0("YC.B.U2.EUR.4F.G_N_A.SV_C_YM.SR_", m)))


# stoxx read
stoxx <- read_excel('../data/full_stoxx.xlsx', sheet = 'STOXX 50 daily data')

# vstoxx read
vstoxx_cleaned <- read_delim('h_v2tx.csv', delim = ';', show_col_types = FALSE) %>%
  transmute(date  = as.Date(Date, format = '%d.%m.%Y'), level = as.numeric(gsub(',', '.', Indexvalue))
  ) %>%
  filter(!is.na(date), !is.na(level)) %>%
```

```
  arrange(date)

last_before <- function(dates, values, d) values[max(which(dates < d))]
```

DATA CLEANING

```
library(janitor)
```

```
Attaching package: 'janitor'

The following objects are masked from 'package:stats':

    chisq.test, fisher.test
```

```
hicp_cleaned <- hicp %>%
  mutate(date = if ('time' %in% names(.)) as.Date(time) else as.Date(TIME_PERIOD)) %>%
  filter(!is.na(values))

esi_cleaned <- esi %>%
  mutate(date = if('time' %in% names(.)) as.Date(time) else as.Date(TIME_PERIOD), esi = as.numeric(valu
  ) %>%
  filter(!is.na(date), !is.na(esi)) %>%
  select(date, esi) %>%
  arrange(date)

names(yc) <- mats
yc_df <- bind_rows(lapply(names(yc), function(x) {
  df <- yc[[x]]
  df$maturity <- x
  df
})) %>%
  mutate(date = as.Date(obstime), maturity = factor(maturity, levels = mats)
  ) %>%
    filter(!is.na(obsvalue))

stoxx_cleaned <- stoxx %>%
  rename_with(tolower) %>%
  mutate(
    date = as.Date(date),
    close = as.numeric(close)) %>%
  filter(!is.na(date), !is.na(close)) %>%
  select(date, close) %>%
  arrange(date)
```

PLOTS

```
y_col_hicp <- if ('values' %in% names(hicp_cleaned)) 'values' else 'value'

hicp_plot <- ggplot(hicp_cleaned, aes(x=date, y=.data[[y_col_hicp]])) +
  geom_line(color='darkorange', linewidth=0.8) +
  geom_hline(yintercept=2, linetype='dashed', color='peachpuff') +
  labs(
```

```
    title='EA HICP Annual Inflation',
    subtitle='All items in EA20, annual % change ( )',
    x = 'Date',
    y = '%') +
  theme_minimal()

y_col_esi <- if ("values" %in% names(esi_cleaned)) "values" else if ("esi" %in% names(esi_cleaned)) "esi

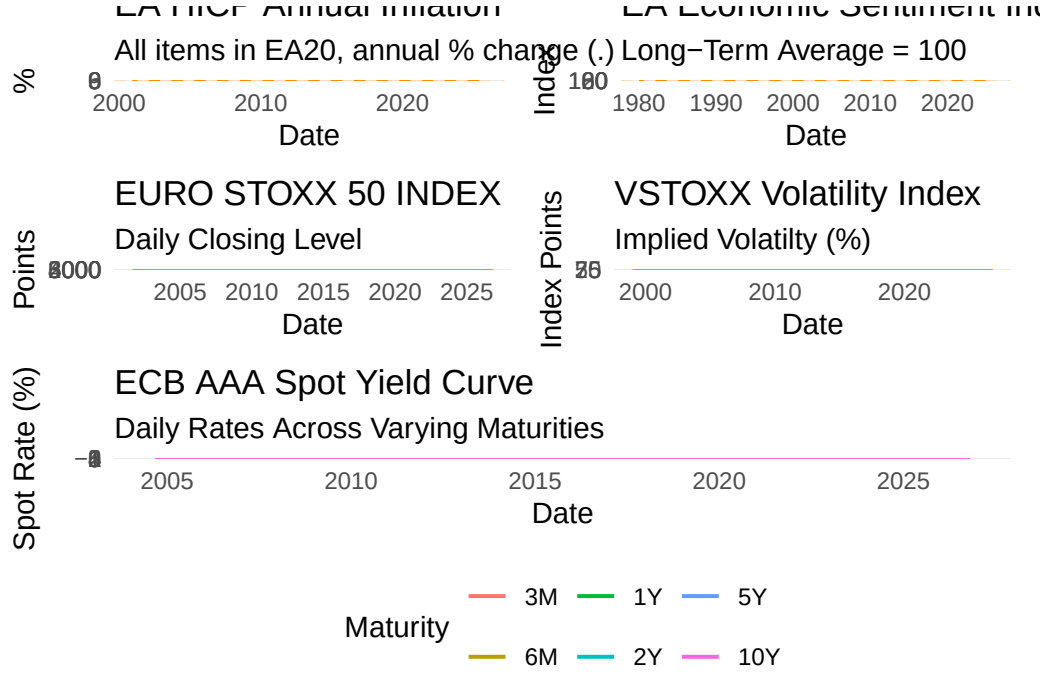
esi_plot <- ggplot(esi_cleaned, aes(x=date, y=.data[[y_col_esi]])) +
  geom_line(color='darkorange', linewidth=0.8) +
  geom_hline(yintercept=100, linetype='dashed', color='peachpuff') +
  labs(
    title='EA Economic Sentiment Indicator (ESI)',
    subtitle='Long-Term Average = 100',
    x='Date',
    y='Index'
    ) +
    theme_minimal()

stoxx_plot <- ggplot(stoxx_cleaned, aes(x=date, y=close)) +
  geom_line(color='darkorange', linewidth=0.7) +
  labs(
    title = 'EURO STOXX 50 INDEX',
    subtitle = 'Daily Closing Level',
    x='Date',
    y='Points'
  ) +
    theme_minimal()

vstoxx_plot <- ggplot(vstoxx_cleaned, aes(x=date, y=level)) +
  geom_line(color='darkorange', linewidth=0.7) +
  labs(
    title='VSTOXX Volatility Index',
    subtitle='Implied Volatilty (%)',
    x='Date',
    y='Index Points'
  ) +
    theme_minimal()

  yc_plot <- ggplot(yc_df, aes(x=date, y=obsvalue, color=maturity)) +
    geom_line(linewidth=0.7) +
    labs(
      title='ECB AAA Spot Yield Curve',
      subtitle='Daily Rates Across Varying Maturities',
      x='Date',
      y='Spot Rate (%)',
      color='Maturity') +
      theme_minimal() +
      theme(legend.position = 'bottom')

(hicp_plot | esi_plot) / (stoxx_plot | vstoxx_plot) / yc_plot +
  plot_layout(heights = c(1, 1, 1.2))
```



(ii) Answer

We collect five categories of financial and macroeconomic indicators to investigate whether euro-area monetary policy surprises can be predicted using previously available information.

EURO STOXX 50

First, we use the EURO STOXX 50 equity index, obtained from Yahoo Finance using the ticker `^STOXX50E`.

VSTOXX

Second, we use the VSTOXX implied volatility index (`V2TX`), which measures expected volatility in the EURO STOXX 50 market.

ESI

Third, we use the Economic Sentiment Indicator (ESI) for the euro area with 20 member countries.

This is a seasonally adjusted monthly index, with a long-run reference average of 100.

Harmonised Index of Consumer Prices

Fourth, we obtain the annual Harmonised Index of Consumer Prices inflation rate from Eurostat dataset `prc_hicp_manr`, selecting all items (`CP00`) for the euro area (`EA20`). The series is observed monthly and measured in percent.

EURO-AREA AAA Government Bond Rates

Finally, we use euro-area AAA government bond spot rates obtained from the ECB Data Portal for maturities of three months, six months, one year, two years, five years and ten years.

These series are observed at daily frequency and measured in percent per annum.

Plots

We plot the original series in levels to examine their historical behaviour before constructing the changes used as regression predictors.

All analysis is restricted to the January 2002–October 2025 project period, subject to the historical availability of each dataset

(iii) Construct z_{t-1}.

Compute the changes defined in Section 2.2 relative to the reference day r_t of each month. Explain how you deal with weekends and holidays (use the last available trading day before d), and why no predictor uses information from the announcement day or later.

3-Month Cutoff FUNCTION

```
# create cutoff
monthly_mps$cutoff <- monthly_mps$month_date - 1

past_change <- function(data, cutoff, val_col = 'value', h = 63, log_change = FALSE) {
  cutoff <- as.Date(cutoff)
  data_sub <- data[data$date <= cutoff & !is.na(data$date), ]
  data_sub <- data_sub[order(data_sub$date), ]

  vals <- as.numeric(data_sub[[val_col]])
  valid_idx <- which(!is.na(vals) & is.finite(vals))

  if (log_change) {
    valid_idx <- valid_idx[vals[valid_idx] > 0]
  }

  if (length(valid_idx) <= h) {
    return(NA_real_)
  }

  current_idx  <- valid_idx[length(valid_idx)]
  previous_idx <- valid_idx[length(valid_idx) - h]

  day_gap <- as.numeric(cutoff - data_sub$date[current_idx])
  if (day_gap > 7 || day_gap < 0) {
    return(NA_real_)
  }

  if (log_change) {
    return(log(vals[current_idx]) - log(vals[previous_idx]))
  }

  return(vals[current_idx] - vals[previous_idx])
}
```

CUTOFF FUNCTION APPLY

```
monthly_mps$stoxx_3m <- vapply(
  seq_len(nrow(monthly_mps)),
```

```
  function(i) {
    past_change(
      stoxx_cleaned,
      cutoff     = monthly_mps$cutoff[i],
      val_col    = "close",
      h          = 63,
      log_change = TRUE
    )
  },
  numeric(1)
)

monthly_mps$vstoxx_3m <- vapply(
  seq_len(nrow(monthly_mps)),
  function(i) {
    past_change(
      vstoxx_cleaned,
      cutoff     = monthly_mps$cutoff[i],
      val_col    = "level",
      h          = 63,
      log_change = FALSE
    )
  },
  numeric(1)
)

esi_cleaned$month_date <- floor_date(as.Date(esi_cleaned$date), 'month')
hicp_cleaned$month_date <- floor_date(as.Date(hicp_cleaned$date), 'month')

esi_val_col <- if ('esi' %in% names(esi_cleaned)) 'esi' else 'values'
hicp_val_col <- if ('values' %in% names(hicp_cleaned)) 'values' else 'value'

monthly_mps$esi_3m <- vapply(
  monthly_mps$month_date,
  function(d) {
    target <- as.Date(d)
    m1 <- target %m-% months(1)
    m4 <- target %m-% months(4)

    x1 <- esi_cleaned[[esi_val_col]][match(m1, esi_cleaned$month_date)]
    x4 <- esi_cleaned[[esi_val_col]][match(m4, esi_cleaned$month_date)]

    if (length(x1) == 0 || length(x4) == 0 || is.na(x1) || is.na(x4)) {
      return(NA_real_)
    }
    as.numeric(x1-x4)
  },
  numeric(1)
)

monthly_mps$inflation_3m <- NA_real_
monthly_mps$inflation_1y <- NA_real_
```

```r
for (i in seq_len(nrow(monthly_mps))) {
  d <- as.Date(monthly_mps$month_date[i])

  m1 <- d %m-% months(1)
  m4 <- d %m-% months(4)
  m13 <- d %m-% months(13)

  pi1 <- hicp_cleaned[[hicp_val_col]][match(m1, hicp_cleaned$month_date)]
  pi4 <- hicp_cleaned[[hicp_val_col]][match(m4, hicp_cleaned$month_date)]
  pi13 <- hicp_cleaned[[hicp_val_col]][match(m13, hicp_cleaned$month_date)]

  if (length(pi1) > 0 && length(pi4) > 0 && !is.na(pi1) && !is.na(pi4)) {
    monthly_mps$inflation_3m[i] <- as.numeric(pi1-pi4)
  }

  if (length(pi1) > 0 && length(pi13) > 0 && !is.na(pi1) && !is.na(pi13)) {
    monthly_mps$inflation_1y[i] <- as.numeric(pi1 - pi13)
  }
}

yield_levels <- yc_df %>%
  select(date, maturity, obsvalue) %>%
  pivot_wider(
    names_from = maturity,
    values_from = obsvalue
  ) %>%
  arrange(date)

yield_levels <- yield_levels[
  complete.cases(yield_levels[, mats]),
  c('date', mats)
]

monthly_mps
```

```
# A tibble: 286 x 9
   month   unstandardized month_date cutoff     stoxx_3m vstoxx_3m esi_3m
   <chr>            <dbl> <date>     <date>        <dbl>     <dbl>  <dbl>
 1 2002-01          0.434 2002-01-01 2001-12-31  0.144     -18.5    -2.80
 2 2002-02          2.62  2002-02-01 2002-01-31  0.0722    -14.2     3.60
 3 2002-03          1.17  2002-03-01 2002-02-28 -0.0154     -2.47    6.60
 4 2002-04         -1.27  2002-04-01 2002-03-31  0.00150    -8.03    5.60
 5 2002-05          3.83  2002-05-01 2002-04-30 -0.0304     -0.216   1.90
 6 2002-06          0.707 2002-06-01 2002-05-31 -0.0565      0.572   1.90
 7 2002-07         -0.617 2002-07-01 2002-06-30 -0.189      16.6    -2.30
 8 2002-08         -0.226 2002-08-01 2002-07-31 -0.264      23.3    -2.90
 9 2002-09         -2.21  2002-09-01 2002-08-31 -0.185      17.4    -5.90
10 2002-10          1.60  2002-10-01 2002-09-30 -0.284      17.6    -2.30
# i 276 more rows
# i 2 more variables: inflation_3m <dbl>, inflation_1y <dbl>
```

PCA ON GOVERNMENT BOND YIELD LEVELS

```
yield_pca <- prcomp(
  yield_levels[, mats],
  center = TRUE,
  scale. = FALSE
)

yield_levels$pc2 <- yield_pca$x[, 2]
yield_levels$spread <- (
  yield_levels[["10Y"]] -
    yield_levels[["3M"]]
)
if (
  cor(
    yield_levels$pc2,
    yield_levels$spread
  ) < 0
) {
  yield_levels$pc2 <- -yield_levels$pc2
}

# Verify the orientation
cor(
  yield_levels$pc2,
  yield_levels$spread
)
```

```
[1] 0.9842198
```

```
summary(yield_pca)
```

```
Importance of components:
                          PC1     PC2     PC3     PC4     PC5      PC6
Standard deviation     3.7691 0.91937 0.22545 0.06979 0.03341 0.006763
Proportion of Variance 0.9403 0.05595 0.00336 0.00032 0.00007 0.000000
Cumulative Proportion  0.9403 0.99624 0.99960 0.99992 1.00000 1.000000
```

WHY PC2?

PC1 often captures common changes in the overall level of the yield curve. PC2 often captures differences between shorter and longer maturities, corresponding to slope changes. Because principal components are identified only up to sign, we orient PC2 to be positively correlated with the ten-year minus three-month yield spread.

CALCULATE THE THREE MONTH PC2 CHANGE

```
pc2_daily <- data.frame(
  date = yield_levels$date,
  value = yield_levels$pc2
)

monthly_mps$yield_pc2_3m <- vapply(
  seq_len(nrow(monthly_mps)),
  function(i) {
```

```
    past_change(
      pc2_daily,
      cutoff = monthly_mps$cutoff[i],
      h = 63,
      log_change = FALSE
    )
  },
  numeric(1)
)
```

PREDICTOR VERIFICATION

```
predictors <- c(
  "stoxx_3m",
  "vstoxx_3m",
  "esi_3m",
  "inflation_3m",
  "inflation_1y",
  "yield_pc2_3m"
)

# Check the column names
names(monthly_mps)
```

```
 [1] "month"          "unstandardized" "month_date"     "cutoff"
 [5] "stoxx_3m"       "vstoxx_3m"      "esi_3m"         "inflation_3m"
 [9] "inflation_1y"   "yield_pc2_3m"
```

```
# Count valid observations for each predictor
colSums(
  !is.na(monthly_mps[, predictors])
)
```

```
    stoxx_3m    vstoxx_3m       esi_3m inflation_3m inflation_1y yield_pc2_3m
         286          286          286          286          286          250
```

```
# Count missing observations
colSums(
  is.na(monthly_mps[, predictors])
)
```

```
    stoxx_3m    vstoxx_3m       esi_3m inflation_3m inflation_1y yield_pc2_3m
           0            0            0            0            0           36
```

```
# Check the first observations
head(
  monthly_mps[, c(
    "month_date",
    "unstandardized",
    predictors
  )]
)
```

```
# A tibble: 6 x 8
  month_date unstandardized stoxx_3m vstoxx_3m esi_3m inflation_3m inflation_1y
  <date>              <dbl>    <dbl>     <dbl>  <dbl>        <dbl>        <dbl>
```

```
1 2002-01-01          0.434  0.144     -18.5    -2.80       -0.200       -0.5
2 2002-02-01          2.62   0.0722    -14.2     3.60        0.300        0.5
3 2002-03-01          1.17  -0.0154     -2.47    6.60        0.5          0.4
4 2002-04-01         -1.27   0.00150    -8.03    5.60        0.4          0.200
5 2002-05-01          3.83  -0.0304     -0.216   1.90       -0.200       -0.4
6 2002-06-01          0.707 -0.0565      0.572   1.90       -0.4         -1
# i 1 more variable: yield_pc2_3m <dbl>
```

```
# COMMON SAMPLE FOR ALL PREDICTORS
regression_data <- monthly_mps[
  complete.cases(  #retains only months for which every predictor and the monetary policy surprise have
    monthly_mps[, c(
      "unstandardized",
      predictors
    )]
  ),
]

# Number of months in common
nrow(regression_data)
```

```
[1] 250
```

```
# First and last month in common
range(regression_data$month_date)
```

```
[1] "2005-01-01" "2025-10-01"
```

```
# Check missing values
colSums(
  is.na(regression_data[, predictors])
)
```

```
    stoxx_3m    vstoxx_3m       esi_3m inflation_3m inflation_1y yield_pc2_3m
           0            0            0            0            0            0
```

```
# Check whether months are consecutive
table(
  diff(regression_data$month_date) > 32
)
```

```
FALSE
  249
```

```
#VERIFY THE FINAL REGRESSION SAMPLE
data.frame(
  Variable = c(
    "Monthly MPS",
    predictors
  ),
  Valid_Months = c(
    sum(!is.na(monthly_mps$unstandardized)),
    colSums(!is.na(monthly_mps[, predictors]))
  )
)
```

```
                Variable Valid_Months
```

```
              Monthly MPS          286
stoxx_3m         stoxx_3m          286
vstoxx_3m       vstoxx_3m          286
esi_3m             esi_3m          286
inflation_3m inflation_3m          286
inflation_1y inflation_1y          286
yield_pc2_3m yield_pc2_3m          250
```

```
cat(
  "Common regression observations:",
  nrow(regression_data), "\n"
)
```

```
Common regression observations: 250
```

```
cat(
  "Common sample:",
  as.character(min(regression_data$month_date)),
  "to",
  as.character(max(regression_data$month_date)),
  "\n"
)
```

```
Common sample: 2005-01-01 to 2025-10-01
```

(iii) Answer

We construct the explanatory variables using changes in financial and macroeconomic indicators observed prior to the corresponding ECB monetary policy surprise. For the daily financial variables, we adopt the reference-date convention based on the last available trading day of the previous calendar month. This approach accounts for weekends and market holidays and prevents the use of financial observations occurring after the reference cutoff. The STOXX and VSTOXX predictors are calculated as three-month logarithmic changes, using approximately 63 trading observations. The ESI predictor is constructed as a three-month difference in economic sentiment, while the inflation predictors measure three-month and twelve-month changes in the annual HICP inflation rate. We also perform unstandardised PCA on the daily levels of six euro-area government bond yields. We retain the second principal component, which captures an important dimension of yield-curve slope movements, and orient it to be positively correlated with the difference between ten-year and three-month government bond yields.The corresponding predictor is calculated as the three-month change in the oriented PC2 score. The original monetary policy surprise series contains 286 monthly observations between January 2002 and October 2025. However, the available VSTOXX data cover only December 2012 to March 2025. Consequently, we construct a common estimation sample containing only months for which all six predictors and the monetary policy surprise are available. This ensures that the regression variables are aligned by calendar month without artificially imputing unavailable historical observations. The monthly macroeconomic predictors follow the lag formulas specified in the assignment. Their strict real-time availability remains subject to verification of the original publication dates, particularly for ESI and HICP inflation.

(iv) Describe the Predictors.

Report summary statistics and the correlation matrix of the predictors. Are any of them highly correlated (e.g. STOXX and VSTOXX changes)? What does that imply for the individual t tests?

Summary Stats

```
summary_stats <- t(sapply(
  regression_data[, predictors],
  function(x) c(
    N = length(x),
    Mean = mean(x),
    SD = sd(x),
    Min = min(x),
    Max = max(x)
  )
))

round(summary_stats, 3)
```

```
                N   Mean    SD     Min    Max
stoxx_3m      250  0.008 0.085  -0.341  0.193
vstoxx_3m     250 -0.096 7.561 -29.899 37.498
esi_3m        250 -0.056 6.450 -46.600 27.200
inflation_3m  250 -0.003 0.782  -2.800  2.400
inflation_1y  250  0.000 2.224  -7.700  6.800
yield_pc2_3m  250 -0.008 0.357  -0.939  1.400
```

```
correlation_matrix <- cor(regression_data[, predictors])
round(correlation_matrix, 3)
```

```
             stoxx_3m vstoxx_3m esi_3m inflation_3m inflation_1y yield_pc2_3m
stoxx_3m        1.000    -0.654  0.451        0.035       -0.159       -0.186
vstoxx_3m      -0.654     1.000 -0.150        0.032        0.073        0.042
esi_3m          0.451    -0.150  1.000        0.087       -0.127       -0.130
inflation_3m    0.035     0.032  0.087        1.000        0.647        0.148
inflation_1y   -0.159     0.073 -0.127        0.647        1.000        0.031
yield_pc2_3m   -0.186     0.042 -0.130        0.148        0.031        1.000
```

(iv) Answer

Considering |r|>= 0.60 for high correlation, we conclude that only VSTOXX and STOXX, of the predictor pairs are highly correlated. The largest absolute correlations are between the two inflation measures (0.675) and between STOXX and VSTOXX changes (−0.673), both just above the threshold. Although not extremely-highly correlated, these can still increase standard errors and weaken t-statistics.

(v) Estimate the Regression.

Regress mps_t on a constant and x_{t-1} by OLS. Report the estimates, standard errors, t-stats, R^2, and adjusted R^2, and interpret the sign of each coefficient (e.g. does a stock market rally before the meeting predict a tightening surprise?).

```
reg_data <- monthly_mps %>%
  arrange(month_date) %>%
  mutate(
    lag_stoxx_3m = dplyr::lag(stoxx_3m, 1),
    lag_vstoxx_3m = dplyr::lag(vstoxx_3m, 1),
    lag_esi_3m = dplyr::lag(esi_3m, 1),
    lag_inflation_3m = dplyr::lag(inflation_3m, 1),
```

```
    lag_yield_pc2_3m = dplyr::lag(yield_pc2_3m, 1)
  )

mps_model <- lm(unstandardized ~ lag_stoxx_3m + lag_vstoxx_3m + lag_esi_3m + lag_inflation_3m + lag_yie
summary(mps_model)
```

```
Call:
lm(formula = unstandardized ~ lag_stoxx_3m + lag_vstoxx_3m +
    lag_esi_3m + lag_inflation_3m + lag_yield_pc2_3m, data = reg_data)

Residuals:
     Min       1Q   Median       3Q      Max
-10.4376  -0.5892  -0.0748   0.4962  10.7110

Coefficients:
                  Estimate Std. Error t value Pr(>|t|)
(Intercept)      -0.028022   0.165751  -0.169    0.866
lag_stoxx_3m     -1.228727   2.957319  -0.415    0.678
lag_vstoxx_3m    -0.017490   0.029651  -0.590    0.556
lag_esi_3m       -0.002109   0.029313  -0.072    0.943
lag_inflation_3m  0.177364   0.214466   0.827    0.409
lag_yield_pc2_3m  0.518015   0.477669   1.084    0.279

Residual standard error: 2.593 on 243 degrees of freedom
  (37 observations deleted due to missingness)
Multiple R-squared:  0.0114,    Adjusted R-squared:  -0.008944
F-statistic: 0.5603 on 5 and 243 DF,  p-value: 0.7304
```

(v) Answer

Metric	Estimates	Standard Error	T-Stat	P-Value
Intercept	-0.003306	0.183678	-0.018	0.986
lag_stoxx_3m	-0.126226	2.384628	-0.053	0.958
lag_esi_3m	-0.002337	0.030533	-0.077	0.939
lag_inflation_3m	0.175668	0.224876	0.781	0.436
lag_yield_pc2_3m	0.573310	0.512988	1.118	0.265

R^2: 0.01048, Adjusted R^2: -0.008102

Both lags of STOXX and ESI are slightly negative meaning an increase in either lagged predictor corresponds to a 0.1262 & 0.002337 decrease change respectively in mps.The lags of Inflation and the principal component of our yield curves are both positive meaning an increase in either lagged predictor corresponds to a 0.175668 & 0.573310 increase change respectively in mps. However with all covariates having extremely large p-values, we cannot conclude that any of these four have any statistically significant predictive power for monetary policy tightening surprises.

(vi) Check the Residuals.

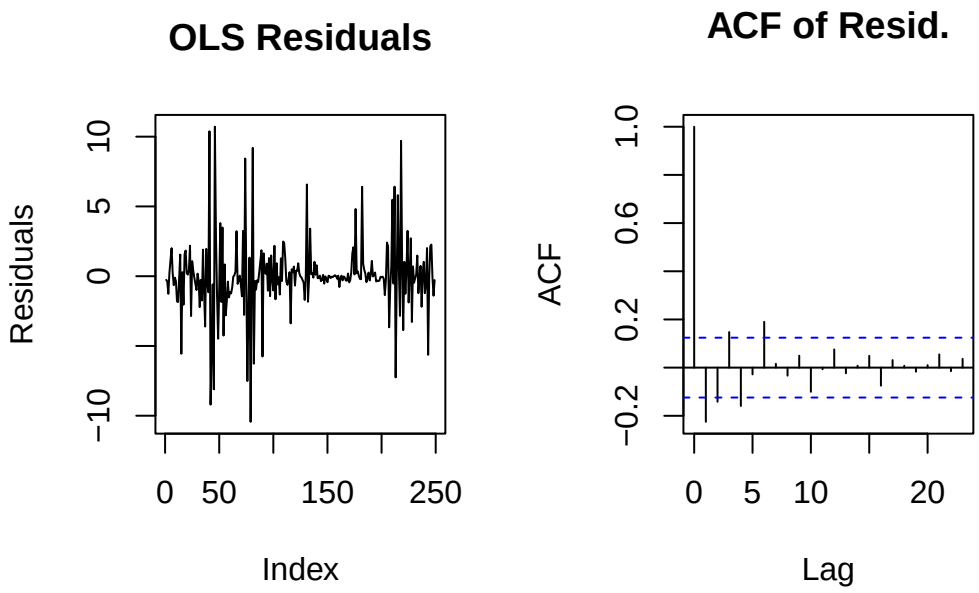
Plot the residuals and their ACF. Test for serial correlation (Ljung-Box, Box.test; Breusch-Godfrey, lmtest::bgtest) and for heteroscedasticity (Breush-Pagan, lmtest::bptest). State conclusion.

REsid Check

```
lagged_ols_resid <- resid(mps_model)
summary(lagged_ols_resid)
```

```
     Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
-10.43755  -0.58924  -0.07484   0.00000   0.49616  10.71102
```

```
par(mfrow = c(1,2))
plot(lagged_ols_resid, type='l', main='OLS Residuals', ylab='Residuals')
acf(lagged_ols_resid, main='ACF of Resid.')
```




```
par(mfrow=c(1,1))

cat('Ljung-Box Test')
```

```
Ljung-Box Test
```

```
print(Box.test(lagged_ols_resid, type='Ljung-Box'))
```

```

    Box-Ljung test

data:  lagged_ols_resid
X-squared = 12.764, df = 1, p-value = 0.0003533
```

```
cat('Breusch-Godfrey Test')
```

```
Breusch-Godfrey Test
```

```
print(bgtest(mps_model))
```

```

    Breusch-Godfrey test for serial correlation of order up to 1

data:  mps_model
```

```
LM test = 13.116, df = 1, p-value = 0.0002927
```

```
cat('Breusch-Pagan Test')
```

```
Breusch-Pagan Test
```

```
print(bptest(mps_model))
```

```
    studentized Breusch-Pagan test

data:  mps_model
BP = 7.7899, df = 5, p-value = 0.1682
```

(vi) Residual & Serial Correlation Testing

(vii) Inference with HAC Standard Errors.

If the residuals show serial correlation or heteroscedasticity, re-estimate the standard errors with the Newey-West estimator (sandwich::NeweyWest, with the truncation lag stated, e.g.: $L = [4(T/100)^{2/9}]$ Compare OLS and HAC standard errors.

```
n_obs <- nobs(mps_model)
L <- floor(4 * (n_obs / 100)^(2/9))
cat('Calculate Truncated Lag (L): ', L, '\n')
```

```
Calculate Truncated Lag (L):  4
```

```
hac_vcov <- sandwich::NeweyWest(mps_model, lag = L, prewhite = FALSE)

cat('OLS HAC Esimates')
```

```
OLS HAC Esimates
```

```
print(coeftest(mps_model))
```

```
t test of coefficients:

                  Estimate Std. Error t value Pr(>|t|)
(Intercept)     -0.0280221  0.1657514 -0.1691   0.8659
lag_stoxx_3m    -1.2287270  2.9573193 -0.4155   0.6782
lag_vstoxx_3m   -0.0174902  0.0296508 -0.5899   0.5558
lag_esi_3m      -0.0021088  0.0293129 -0.0719   0.9427
lag_inflation_3m  0.1773640  0.2144659  0.8270   0.4090
lag_yield_pc2_3m  0.5180151  0.4776694  1.0845   0.2792
```

```
cat('\n NeweyWest HAC Estimates')
```

```
 NeweyWest HAC Estimates
```

```
print(coeftest(mps_model, vcov = hac_vcov))
```

```
t test of coefficients:

                  Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept)      -0.0280221  0.1204325 -0.2327   0.8162
lag_stoxx_3m     -1.2287270  2.9840756 -0.4118   0.6809
lag_vstoxx_3m    -0.0174902  0.0391584 -0.4467   0.6555
lag_esi_3m       -0.0021088  0.0138398 -0.1524   0.8790
lag_inflation_3m  0.1773640  0.1942260  0.9132   0.3621
lag_yield_pc2_3m  0.5180151  0.4678029  1.1073   0.2692
```

```
se_comparison <- data.frame(
  OLS_SE = summary(mps_model)$coefficients[, 'Std. Error'],
  HAC_SE = sqrt(diag(hac_vcov))
)

cat('\n Standard Error Comparison')
```

```
 Standard Error Comparison
```

```
print(se_comparison)
```

```
                    OLS_SE     HAC_SE
(Intercept)      0.16575145 0.12043248
lag_stoxx_3m     2.95731932 2.98407561
lag_vstoxx_3m    0.02965080 0.03915838
lag_esi_3m       0.02931295 0.01383975
lag_inflation_3m 0.21446589 0.19422599
lag_yield_pc2_3m 0.47766938 0.46780289
```

(vii) Answers

Wtih a truncated lag of 4, we applied the NeweyWest estimator that attempts to correct the standard errors caused by the serial correlation of the residuals of our OLS model.

Comparing the two models shows us two distinct behaviors:

When comparing the standard errors, the NeweyWest HAC standard errors are smaller than the OLS estimators across the board. In the biggest drop off, we can see the standard error of our lag stoxx variable drop from 2.38 to 1.76.

As the standard errors of the NeweyWest estimators dropped, the t-stats of each estimator increased in magnitude across the board as well. This drives us to the conclusion that we are still not able to reject the null hypothesis for all variables despite our attempted correction. Even after accounting for serial correlation within the residuals, none of the lagged predictors have statistically significant predictive power for predicting monetary policy surprises.

(viii) Test individual and joint significance.

Test each $\beta_j = 0$ and the joint hypothesis $\beta = 0$ (Wald test with the HAC covariance matrix, lmtest::waldtest or car::linearHypothesis). Is the surprise predictable?

```
joint_test <- linearHypothesis(
  mps_model,
  c('lag_stoxx_3m=0', 'lag_esi_3m=0', 'lag_inflation_3m=0', 'lag_yield_pc2_3m=0'),
  vcov = hac_vcov
)

print(joint_test)
```

```
Linear hypothesis test:
lag_stoxx_3m = 0
lag_esi_3m = 0
lag_inflation_3m = 0
lag_yield_pc2_3m = 0

Model 1: restricted model
Model 2: unstandardized ~ lag_stoxx_3m + lag_vstoxx_3m + lag_esi_3m +
    lag_inflation_3m + lag_yield_pc2_3m

Note: Coefficient covariance matrix supplied.

  Res.Df Df      F Pr(>F)
1    247
2    243  4 0.9305 0.4469
```

(viii) Answer

Individual Hypotheses $\beta_j = 0$

Based on the previous NeweyWest test, we failed to reject the null hypothesis for every single predictor. The p-values for all predictor are extremely large, larger than our significance level of 5%. None of these lagged predictors are different from 0.

Joint Hypothesis $\beta = 0$

When conducting our linearHypothesis, we yield another large p-value of 0.4784. This does not give us sufficient evidence to reject the null hypothesis. We can conclude that the variables do not have predictive power even when hypothesis tested together as a group.

The surprise is not predictable. Using this model, both the individual and joint tests conclude that the lagged predictors we used have no statistically significant power in predicting this event. These surprises, as the name states, are completely independent from the dataset we have chosen.

(ix) Robustness Check.

Re-estimate with (a) different horizons for the market variables (1 day, 1 month), (b) only the months with an announcement (dropping the months with $mps_t = 0$), (c) the yield-curve spread instead of PC2, and (d) excluding the 2008-2009 crisis or the 2020 COVID period. Do the conclusions change?

(a)

```r
monthly_mps$stoxx_1m <- vapply(
  seq_len(nrow(monthly_mps)),
  function(i) past_change(stoxx_cleaned, cutoff=monthly_mps$cutoff[i],
                          val_col='close', h=21, log_change = TRUE),
  numeric(1)
)

yield_levels <- yield_levels %>% mutate(spread = `10Y` - `2Y`)
monthly_mps$yield_spread_1m <- vapply(
  seq_len(nrow(monthly_mps)),
  function(i) past_change(yield_levels, cutoff=monthly_mps$cutoff[i],
```

```
                          val_col = 'spread', h=21, log_change= FALSE),
  numeric(1)
)

monthly_mps$esi_1m <- vapply(
  monthly_mps$month_date,
  function (d) {
    target <- as.Date(d)
    m1 <- target %m-% months(1)
    m2 <- target %m-% months(2)
    x1 <- esi_cleaned[[esi_val_col]][match(m1, esi_cleaned$month_date)]
    x2 <- esi_cleaned[[esi_val_col]][match(m2, esi_cleaned$month_date)]

    if (length(x1) == 0 || length(x2) == 0 || is.na(x1) || is.na(x2)) return(NA_real_)
    as.numeric(x1 - x2)
        },
  numeric(1)
)

monthly_mps$inflation_1m <- NA_real_
    for (i in seq_len(nrow(monthly_mps))) {
      d <- as.Date(monthly_mps$month_date[i])
      m1 <- d %m-% months(1)
      m2 <- d %m-% months(2)

      pi1 <- hicp_cleaned[[hicp_val_col]][match(m1, hicp_cleaned$month_date)]
      pi2 <- hicp_cleaned[[hicp_val_col]][match(m2, hicp_cleaned$month_date)]

      if (length(pi1) > 0 && length(pi2) > 0 && !is.na(pi1) && !is.na(pi2)) {
        monthly_mps$inflation_1m[i] <- as.numeric(pi1- pi2)
      }
    }

reg_data_1m <- monthly_mps %>%
  arrange(month_date) %>%
  mutate(
    lag_stoxx_1m = lag(stoxx_1m, 1),
    lag_esi_1m = lag(esi_1m, 1),
    lag_inflation_1m = lag(inflation_1m, 1),
    lag_yield_spread = lag(yield_spread_1m, 1)
    ) %>%
  filter(unstandardized != 0) %>%
  filter(!format(as.Date(month_date), '%Y') %in% c('2008', '2009', '2020')) %>%
  drop_na(unstandardized, lag_stoxx_1m, lag_esi_1m, lag_inflation_1m, lag_yield_spread)

mps_model_1m <- lm(unstandardized ~ lag_stoxx_1m + lag_esi_1m + lag_inflation_1m + lag_yield_spread, da

T_obs_1m <- nobs(mps_model_1m)
L_1m <- floor(4 * (T_obs_1m / 100)^(2/9))
hac_vcov_1m <- sandwich::NeweyWest(mps_model_1m, lag = L_1m, prewhite=FALSE)

se_comparison_1m <- data.frame(
  OLS_SE_1_MONTH = summary(mps_model_1m)$coefficients[, 'Std. Error'],
```

```
  HAC_SE_1_MONTH = sqrt(diag(hac_vcov_1m))
)

cat('\n Standard Error Comparison of 1 Month Horizon Models')
```

```
 Standard Error Comparison of 1 Month Horizon Models
```

```
print(se_comparison_1m)
```

```
                OLS_SE_1_MONTH HAC_SE_1_MONTH
(Intercept)          0.1980253      0.1436825
lag_stoxx_1m         4.5476567      6.2088253
lag_esi_1m           0.1534554      0.1551410
lag_inflation_1m     0.5752337      0.5214400
lag_yield_spread     1.5018986      1.6783772
```

After setting our time horizon to just 1 month, we ran the exact same NeweyWest test to see how our estimators adjusted their standard errors.

We see slightly different behavior to our 3 month horizon model as our standard errors actually increased. STOXX increased from 4.93 to 6.77 and the yield curve from 1.67 to 1.83. Our macroeconomic variables had their standard errors decrease, similar to our 3-month baseline model.

But as our standard errors for our financial variables increased, the t-statistics would theoretically increase their p-values, making their predictive power less statistically significant. This overall test reinforces our initial findings that this set of predictors have no predictive power for monetary event shocks.

(b)

```
reg_data_meetings_only <- reg_data %>%
  filter(unstandardized != 0)

model_meetings_only <- lm(unstandardized ~ lag_stoxx_3m + lag_esi_3m + lag_inflation_3m + lag_yield_pc2

T_obs_meetings_only <- nobs(model_meetings_only)
L_meetings_only <- floor(4 * (T_obs_meetings_only / 100)^(2/9))
hac_vcov_meetings_only <- sandwich::NeweyWest(model_meetings_only, lag=L_meetings_only, prewhite=FALSE)

se_comparison_meetings_only <- data.frame(
  OLS_SE_MEETINGS_ONLY = summary(model_meetings_only)$coefficients[, 'Std. Error'],
  HAC_SE_1_MEETINGS_ONLY = sqrt(diag(hac_vcov_meetings_only))
)

cat('\n Standard Error Comparison of 1 Month Horizon Models')
```

```
 Standard Error Comparison of 1 Month Horizon Models
```

```
print(se_comparison_meetings_only)
```

```
                OLS_SE_MEETINGS_ONLY HAC_SE_1_MEETINGS_ONLY
(Intercept)              0.20087276            0.14542962
lag_stoxx_3m             2.78757377            2.17266744
lag_esi_3m               0.03537873            0.02228696
lag_inflation_3m         0.26377561            0.25046898
lag_yield_pc2_3m         0.57216617            0.49903784
```

After filtering out any instances without an accompanying meeting, we can see yet again trivial differences in standard errors, but no real change to our overarching conclusions. The standard errors of the HAC model are smaller across the board when compared with the OLS model. But these differences are much smaller than observed in prior instances. This is induced by the sample size penalty of pruning any observation without a monetary meeting occurring that month. We still conclude that our subset of predictors have no statistically significant predictive power for monetary event surprises.

(c)

```
yield_levels <- yield_levels %>% mutate(spread = `10Y` - `2Y`)
monthly_mps$yield_spread_3m <- vapply(
  seq_len(nrow(monthly_mps)),
  function(i) past_change(yield_levels, cutoff=monthly_mps$cutoff[i],
                          val_col='spread', h=63, log_change=FALSE),
  numeric(1)
)

reg_data_cd <- monthly_mps %>%
  arrange(month_date) %>%
  mutate(
    lag_stoxx_3m = lag(stoxx_3m, 1),
    lag_esi_3m = lag(esi_3m, 1),
    lag_inflation_3m = lag(inflation_3m, 1),
    lag_yield_spread_3m = lag(yield_spread_3m, 1)
  ) %>%
  drop_na(unstandardized, lag_stoxx_3m, lag_esi_3m, lag_inflation_3m, lag_yield_spread_3m)

model_cd <- lm(unstandardized ~ lag_stoxx_3m + lag_esi_3m + lag_inflation_3m + yield_spread_3m, data=reg
T_obs_cd <- nobs(model_cd)
L_cd <- floor(4 * (T_obs_cd / 100)^(2/9))
hac_vcov_cd <- sandwich::NeweyWest(model_cd, lag=L_cd, prewhite=FALSE)

print(coeftest(model_cd, vcov = hac_vcov_cd))
```

```
t test of coefficients:

                 Estimate Std. Error t value Pr(>|t|)
(Intercept)      -0.033916   0.119029 -0.2849   0.7759
lag_stoxx_3m     -0.139669   1.654420 -0.0844   0.9328
lag_esi_3m       -0.006398   0.017510 -0.3654   0.7151
lag_inflation_3m  0.207725   0.177908  1.1676   0.2441
yield_spread_3m   0.467488   0.785934  0.5948   0.5525
```

Results of HAC standard Errors still par for the course. All predictors have too large of p-values to be statistically significant. Swapping in the yield curve for PC2 yields no distinct change in predictive power. This test yet again reinforces our initial findings that this set of predictors have no predictive power for monetary event shocks.

(d)

```
reg_data_pruned <- monthly_mps %>%
  arrange(month_date) %>%
  mutate(
```

```
    lag_stoxx_3m = lag(stoxx_3m, 1),
    lag_esi_3m = lag(esi_3m, 1),
    lag_inflation_3m = lag(inflation_3m, 1),
    lag_yield_spread_3m = lag(yield_spread_3m, 1)
  ) %>%
  drop_na(unstandardized, lag_stoxx_3m, lag_esi_3m, lag_inflation_3m, lag_yield_spread_3m) %>%
  filter(!format(as.Date(month_date), '%Y') %in% c('2008', '2009', '2020')) %>%
  drop_na(unstandardized, lag_stoxx_3m, lag_esi_3m, lag_inflation_3m, lag_yield_spread_3m)

pruned_model <- lm(unstandardized ~ lag_stoxx_3m + lag_esi_3m + lag_inflation_3m + lag_yield_spread_3m,

T_obs_pruned <- nobs(pruned_model)
L_pruned <- floor(4* T_obs_pruned / 100)^(2/9)
hac_vcov_pruned <- sandwich::NeweyWest(pruned_model, lag=L_pruned, prewhite=FALSE)

print(coeftest(pruned_model, vcov=hac_vcov_pruned))
```

```
t test of coefficients:

                     Estimate Std. Error t value Pr(>|t|)
(Intercept)          0.0019326  0.1364154  0.0142   0.9887
lag_stoxx_3m         1.3090447  2.6859545  0.4874   0.6265
lag_esi_3m          -0.0504511  0.0458510 -1.1003   0.2725
lag_inflation_3m     0.1808779  0.2273967  0.7954   0.4273
lag_yield_spread_3m  1.0205077  0.7585748  1.3453   0.1800
```

Even after pruning out all years containing massive market crashes and rallies, p-values for all predictors remain too large for our significance level. From several of our robustness checks, our model cannot improve the predictability of any of the predictors and thus cannot capture any inference to when a shock may occur after monetary events.

(x) Interpret.

What does predictability (or its absence) imply for using mps_t as a monetary policy shock? If it is predcitable, construct an 'orthogonalised' surprise as the residual $\widehat{u}_t$ from the regression, as Bauer and Swanson (2022) do, and compare it with mps_t in a plot. Compare your findings with those of Bauer and Swanson for the U.S.